

REVISÃO - COMPUTATIONAL INTELLIGENCE & ALGORITHM OPTIMIZATION

1. Módulos & Espaços de Busca

Problemas de otimização pertencem a classes de complexidade **NP-Difícil** (*NP-Hard*). À medida que o tamanho da entrada (N) cresce, o número de combinações possíveis sofre uma **explosão combinatória** $O(2^N)$ ou $O(N!)$. Buscar uma solução exata testando todas as alternativas via força bruta torna-se matematicamente inviável dentro do tempo de vida do universo.

Imagina que você precisa encontrar o caminho mais curto para um entregador passar por 50 cidades. Se tentar testar todas as ordens possíveis no computador, o cálculo demoraria bilhões de anos. Em vez de testar todas as combinações (Força Bruta), usamos as Meta-heurísticas: estratégias inteligentes para encontrar boas soluções em problemas complexos.

Imagine que você está preso no topo de uma cordilheira coberta por uma névoa densa e seu objetivo é encontrar o ponto mais baixo do vale (**Mínimo Global**). Você só consegue enxergar um metro à sua frente. Se você apenas andar para baixo (Busca Local), pode cair em um pequeno buraco (**Mínimo Local**) e achar que chegou ao fim do vale.

As meta-heurísticas são como ferramentas especiais, como bússolas, que te ajudam a escapar desses vales para achar o local mais profundo.

2. Busca Local vs. Busca Global: Exploration & Exploitation

O sucesso de qualquer algoritmo evolutivo depende do equilíbrio perfeito entre duas forças opostas:

- **Exploration (Exploração / Diversificação):** Capacidade de varrer regiões inéditas do espaço de busca para evitar a convergência prematura: é o perfil do explorador: ele viaja para lugares desconhecidos procurando ouro. Ele pode achar algo incrível ou voltar de mãos vazias.
- **Exploitation (Exploração / Intensificação):** Capacidade de concentrar a busca nas vizinhanças de soluções promissoras já encontradas para fazer o "ajuste fino": é o perfil do garimpeiro minucioso: ele já sabe que há ouro naquele rio e passa a peneira milímetro por milímetro para extrair a maior quantidade possível.

Se você só faz *Exploration*, fica pulando de galho em galho e nunca lapida uma boa resposta. Se você só faz *Exploitation*, você passa a vida inteira refinando a melhor rota de uma cidade horrível sem perceber que a cidade vizinha tinha um caminho 100 vezes melhor.

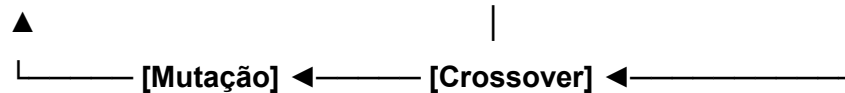
3. Algoritmos Genéticos (AG): Seleção Natural no Código

Inspirado na Teoria da Evolução de Darwin. Uma população de soluções candidata (Cromossomos/Indivíduos) evolui ao longo de gerações. Cada indivíduo possui um valor de avaliação (Fitness/Aptidão).

Os operadores evolutivos fundamentais são:

1. **Seleção (Roleta ou Torneio):** Seleciona os melhores com maior probabilidade (pressão seletiva).
 2. **Cruzamento (Crossover):** Combina material genético de dois pais para gerar descendentes.
 3. **Mutação:** Altera aleatoriamente um gene com uma probabilidade baixa (p_m) para introduzir diversidade genética.
- Criamos centenas de soluções "rascunho". Testamos todas e damos uma nota (*Fitness*) para cada uma.
 - As melhores soluções ganham o direito de se "cruzar" e gerar soluções filhas, herdando as boas qualidades dos pais.
 - De vez em quando, aplicamos uma "mutação" (uma mudança aleatória) para ver se surge uma ideia genial inédita.

[População Inicial] —► [Avalia Fitness] —► [Seleção por Torneio]



4. Otimização por Enxame de Partículas (PSO)

O **PSO** (*Particle Swarm Optimization*) inspira-se no comportamento social de bandos de pássaros ou cardumes. Cada partícula representa uma solução e vai pelo espaço de busca ajustando sua velocidade com base em três vetores:

1. **Inércia** (w): Tendência de manter a direção atual do voo.
2. **Memória Cognitiva** ($c_1 \cdot r_1$): A atração em direção à melhor posição que a *própria* partícula já visitou (p_{best}).
3. **Memória Social** ($c_2 \cdot r_2$): A atração em direção à melhor posição que *qualquer* partícula do enxame já visitou (g_{best}).

$$\vec{V}_i^{t+1} = \underbrace{w \cdot \vec{V}_i^t}_{\text{Inércia}} + \underbrace{c_1 r_1 (\vec{pbest}_i - \vec{X}_i^t)}_{\text{Atração Pessoal (Cognitiva)}} + \underbrace{c_2 r_2 (\vec{gbest} - \vec{X}_i^t)}_{\text{Atração Coletiva (Social)}}$$

Imagine 20 pessoas procurando um tesouro enterrado em uma praia gigante de olhos vendados, mas conectadas por rádio.

Cada pessoa se move pensando: "Vou continuar andando na direção que já estou indo (inércia), mas vou puxar um pouco em direção ao ponto mais fundo que eu pessoalmente já achei (cognitivo), e vou me puxar bastante na direção do ponto mais fundo que o meu colega do rádio achou (social)".

5. Algoritmos de Colônia de Formigas (ACO) e Algoritmos Meméticos

1. **ACO (Ant Colony Optimization):** Projetado para otimização em grafos (rotas, redes). Baseia-se no depósito e na evaporação de **Feromônio** τ (tau minúsculo) combinados com a **Visibilidade** (η (eta) = $\frac{1}{\text{custo}}$) - quanto menor o custo, maior a visibilidade. Caminhos curtos acumulam feromônio mais rápido antes que ele evapore, atraindo o restante da colônia.

Ou seja:

τ : "As outras formigas já descobriram que este caminho funciona."

η : "Este caminho parece bom porque o custo é baixo."

Na regra de escolha do ACO, os dois podem ser combinados:

$$P_{ij} \propto \tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$$

Assim, a formiga considera tanto o que a colônia já aprendeu quanto o que o custo atual indica.

No contexto do ACO, τ (tau) representa a quantidade ou intensidade de feromônio depositada em uma aresta/caminho.

A **visibilidade** é relacionada ao custo do caminho: quanto menor o custo c_{ij} , maior a visibilidade η_{ij}

2. **Algoritmos Meméticos:** combinam uma busca global, realizada por uma meta-heurística como AG ou ACO, com uma busca local, na qual cada solução encontrada é refinada para melhorar sua qualidade antes de seguir para a próxima etapa do algoritmo.

Na ACO Uma formiga anda pela floresta soltando um cheiro no chão (feromônio). Se o caminho for um atalho, ela vai e volta rápido, acumulando muito cheiro.

Mas a **Formiga Memética** é esperta: assim que ela encontra um caminho, ela para, olha em volta, corta uma volta desnecessária no mato (*Busca Local*) e só depois avisa as outras formigas do grupo.

Resumo comparativo

Algoritmo	Mecanismo Principal	Ponto Forte	Indicado Para
Algoritmo Genético (AG)	Seleção, Crossover e Mutação	Ampla exploração (<i>Exploration</i>)	Problemas combinatórios de seleção/mochila

Algoritmo	Mecanismo Principal	Ponto Forte	Indicado Para
PSO (Enxame)	Vetores de Inércia + Pessoal + Social	Velocidade e ajuste fino (<i>Exploitation</i>)	Otimização contínua de parâmetros e limiares
ACO (Formigas)	Trilha de Feromônio + Evaporação	Roteamento dinâmico e grafos	Menor caminho, redes de tráfego, caixeiro viajante
Memético	Meta-heurística + Busca Local	Alta precisão e convergência rápida	Problemas complexos com orçamento computacional